Dr. Dobb's Digest November 2009

Prefer Structured Lifetimes: Local, Nested, Bounded, Deterministic

What's good for the function and the object is also good for the thread, the task, and the lock

By Herb Sutter

Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

There was a time when it was a novel idea that function calls should obey proper nesting, meaning that the lifetime of a called function should be a proper subset of the lifetime of the function that called it:

```
void f() {
  //
  g();        // jump to function g here and then
  // return from function g and continue here!
  //
}
```

"Eureka!" said Edsger Dijkstra. "Function **g**'s execution occurs entirely within that of function **f**. Boy, that sure seems easier to reason about than jumping in and out of random subroutines with unstructured **goto**s. I wonder what to call this idea. There seems to be inherent structure to it. Hmm, I bet I could build a deterministic and efficient model of 'stack local variables' around it too and maybe I should write a letter" (I paraphrase.) [1]

That novel idea begat the discipline of structured programming. This was a huge boon to programming in general, because structured code was naturally localized and bounded so that parts could be reasoned about in isolation, and entire programs became more understandable, predictable, and deterministic. It was also a huge boon to reusability and a direct enabler of reusable software libraries as we know them today, because structured code made it much easier to treat a call tree (here, **f** and **g** and any other functions they might in turn call) as a distinct unit -- because now the call graph really could be relied upon to be a tree, not the previously usual plate of "**goto** spaghetti" that was difficult to isolate and disentangle from its original environment. The structuredness that let any call tree be designed, debugged, and delivered as a unit has worked so well, and made our code so much easier to write and understand, that we still apply it rigorously today: In every major language, we just expect that "of course" function calls on the same thread should still logically nest by default, and doing anything else is hardly imaginable.

That's great, but what does it have to do with concurrency?

A Tale of Three Kinds of Lifetimes

In addition to the function lifetimes we've just considered, Table 1 shows three more kinds of lifetimes -- of objects, of threads or tasks, and of locks or other exclusive resource access -- and for each one lists some structured examples, unstructured examples, and the costs of the unstructured mode.

	Object lifetimes	Thread/task lifetimes	Lock lifetimes
Structured Examples	Function local vars: • C++ stack scope • C# using blocks • Java dispose pattern By-value parameters By-value nested objects	Recursive/data decomposition: • Subrange per nested stack call • Partitioned sub-ranges, parallel calls Join-before-return, internal parallelism	Scoped locking: • C++ lock_guard • C# lock blocks • Java synchronized blocks Release-before-return
Unstructured Examples	Global objects Heap/lib allocation	Spawn "new thread()" Spawn process	Explicit "lock" call, with no automatic unlock or "finally unlock" call
Unstructured Costs and Drawbacks	Nondeterministic finalization timing Allocation overhead Tracking overhead to not use after delete: • GC • smart pointers Ownership cycles	Nondeterministic execution/join timing Allocation overhead Tracking overhead to not use task after EOT, join task before EOP Ownership/waiting cycles	Nondeterministic lock acq/rel ordering Allocation overhead Tracking overhead to avoid deadlock, priority inversion, ... Waiting/blocking cycles (deadlock)

Table 1

For familiarity, let's start with object lifetimes (left column). I'll dwell on it a little, because the fundamental issues are the same as in the next two columns even though those more directly involve concurrency.

In the mainstream OO languages, a structured object lifetime begins with the constructor, and ends with the destructor (C++) or **dispose** method (C# and Java) being called before returning from the scope or function in which the object was created. The bounded, nested lifetime means that cleanup of a structured object is deterministic, which is great because there's no reason to hold onto a resource longer than we need it. The object's cleanup is also typically much faster, both in itself and in its performance impact on the rest of the system. [2] In all of the popular mainstream languages, programmers directly use structured function-local object lifetimes where possible for code clarity and performance:

- In some languages, we get to express the structured lifetime using a language feature, such as stack-based or by-value nested member objects in C++, and **using** blocks in C#.
- In other languages, we use a programming idiom or convention, such as the **try/finally** dispose pattern in Java, and explicit dispose-chaining (to have our object's **dispose** also call **dispose** on other objects exclusively owned by our object, the equivalent of by-value nested member objects) in both C# and Java.

Unstructured, non-local object lifetimes happen with global objects or dynamically allocated objects, which include objects your program may explicitly allocate on the heap and objects that a library you use may allocate on demand on your behalf. Even basic allocation costs more for unstructured, heap-based objects than for structured, stack-based ones. Objects with unstructured lifetimes also require more bookkeeping -- either by you such as by using smart pointers, or by the system such as with garbage collection and finalization. Importantly, note that C# and Java GC-time finalization [3] is not the same as disposing, and you can only do a restricted set of things in a finalizer. For example, in object **A**'s finalizer it's not generally safe to use any other finalizable object **B**, because **B** might already have been finalized and no longer be in a usable state. Lest we be tempted to sneer at finalizers, however, note also that C++'s shutdown rules for global/static objects, while somewhat more deterministic, are intricate bordering on arcane and require great care to use reliably. So having an unstructured lifetime really does have wide-ranging consequences to the robustness and determinism of your program, particularly when it's time to release resources or shut down the whole system.

Speaking of shutdown: Have you ever noticed that program shutdown is inherently a deeply mysterious time? Getting orderly shutdown right requires great care, and the major root cause is unstructured lifetimes: the need to carefully clean up objects whose lifetimes are not deterministically nested and that might depend on each other. For example, if we have an open SQLConnection object, on the one hand we must be sure to **Close()** or **Dispose()** it before the program exits; but on the other hand, we can't do that while any other part of the program might still need to use it. The system usually does the heavy lifting for us for a few well-known global facilities like console I/O, but we have to worry about this ourselves for everything else.

This isn't to say that unstructured lifetimes shouldn't be used; clearly, they're frequently necessary. But unstructured lifetimes shouldn't be the default, and should be replaced by structured lifetimes wherever possible. Managing nondeterministic object lifetimes can be hard enough in sequential code, and is more complex still in concurrent code.

Enter Concurrency

All of the issues and costs associated with unstructured object lifetimes apply in full force to concurrency. And not only do we see "similar" issues and costs arise in the case of unstructured concurrency, but often we see the very same ones.

Threads and tasks have unstructured lifetimes by default on most systems, and therefore by default non-local, non-nested, unbounded, and nondeterministic. That's the root cause of most of threads' major problems. [4] As with unstructured object lifetimes, to manage unstructured thread and task lifetimes we need to perform tracking to make sure we don't try to wait for or communicate with a task after that task has already ended (similar to use-after-delete or use-after-dispose issues with objects); and to join with a thread before the end of the program to avoid risking undefined behavior on nearly all platforms (similar to unstructured "eventual" finalization at shutdown time). Unstructured threads and tasks also incur extra over-heads [5], such as extra blocking when we attempt to join with multiple pieces of work; they are also more likely to have ownership cycles that have to be detected and cleaned up, and similarly for waiting cycles and even deadlock (e.g., when two threads wait for each other's messages).

Structured thread and task lifetimes are certainly possible. You just have to make it happen by applying a dose of discipline when they aren't structured by default: A function that spawns some asynchronous work has to be careful to also join with that work before the function itself returns, so that the lifetime of the spawned work is a subset of the invoking function's lifetime and there are no outstanding tasks, no pending futures, no lazily deferred side effects that the caller will assume are already complete. Otherwise, chaos can result, as we'll see when we analyze an example of this in the next section.

With locks, the stakes are higher still to keep lifetimes bounded -- the shorter the better -- and deterministic. Deadlocks happen often enough when lock lifetimes all are structured, and being structured isn't enough by itself to avoid deadlock [6]; but toying with unstructured locking is just asking for a generous helping of latent deadlocks to be sprinkled throughout your application. Unstructured lock lifetimes also incur the other issues common to all unstructured lifetimes, including tracking overhead to manage the locks, and performance overhead from excessive waiting/blocking that can cause throttling and delay even when there isn't an outright deadlock.

Given the stakes, it's no surprise that every major OO language offers direct language support for bounded lock lifetimes where the lock is automatically released at the end of the scope or the function:

- C++0x has **std::lock_guard**, which is used in a way that just leverages C++'s bounded stack-based variable rules.
- C# has **lock** blocks, and can also leverage its **using** blocks with lock types that implement **IDisposable**.
- Java has **synchronized** blocks, and can leverage the try/**finally** dispose pattern for disposable lock objects.

These tools exist for a reason. Strongly prefer using these structured lifetime mechanisms for scoped locking whenever possible, instead of following the dark path of unstructured locking by explicit standalone calls to Lock functions without at least a finally to Unlock at the end of the scope.

Having all that in mind, we'll turn to an example that illustrates a simple but common mistake that has its root in unstructuredness.

An Example, With a Common Initial Mistake

Let's take a fresh look at the same Quicksort example we considered briefly last month. [7] Here is a simplified traditional sequential implementation of quicksort:

```
// Example 1: Sequential quicksort
void Quicksort( Iter first, Iter last ) {
  // If the range is small, another sort is usually faster.
  if( distance( first, last ) < limit ) {
    OtherSort( first, last );
    return;
  }
  // 1. Pick a pivot position somewhere in the middle.
  // Move all elements smaller than the pivot value to
  // the pivot's left, and all larger elements to its right.
  Iter pivot = Partition( first, last );
  // 2. Recurse to sort the subranges.
  Quicksort( first, pivot );
  Quicksort( pivot, last );
}
```

How would you parallelize this function? Here's one attempt, in Example 2 below. It gets several details right: For good performance, it correctly reverts to a synchronous sort for smaller ranges, just like a good synchronous implementation reverts to a non-quicksort for smaller ranges; and it keeps the last chunk of work to run synchronously on its own thread to avoid a needless extra context switch and preserve better locality.

```
// Example 2: Parallelized quicksort, take 1 (flawed)
void Quicksort( Iter first, Iter last ) {
  // If the range is small, synchronous sort is usually faster.
  if( distance( first, last ) < limit ) {
    OtherSort( first, last );
    return;
  }
  // 1. Pick a pivot position somewhere in the middle.
  // Move all elements smaller than the pivot value to
  // the pivot's left, and all larger elements to its right.
  Iter pivot = Partition( first, last );
  // 2. Recurse to sort the subranges. Run the first
  // asynchronously, while this thread continues with
  // the second.
  pool.run(  [=]{ Quicksort( first, pivot ); }  );
  Quicksort( pivot, last );
}
```

Note: **pool.run(x)** just means to create task **x** to run in parallel, such as on a thread pool; and **[=]{ f; }** in C++, or **()=>{ f(); }** in C#, is just a convenient lambda syntax for creating an object that can be executed (in Java, just write a runnable object by hand).

But now back to the main question: What's wrong with this code in Example 2? Think about it for a moment before reading on.

What's Wrong, and How To Make It Right

The code correctly executes the subrange sorts in parallel. Unfortunately, what it doesn't do is encapsulate and localize the concurrency, because this version of the code doesn't guarantee that the subrange sorts will be complete before it returns to the caller. Why not? Because the code failed to join with the spawned work. As illustrated in Figure 1, subtasks can still be running while the caller may already be executing beyond its call to Quicksort. The execution is unstructured; that is, unlike normal structured function calls, the execution of subtasks does not nest properly inside the execution of the parent task that launched them, but is unbounded and nondeterministic.

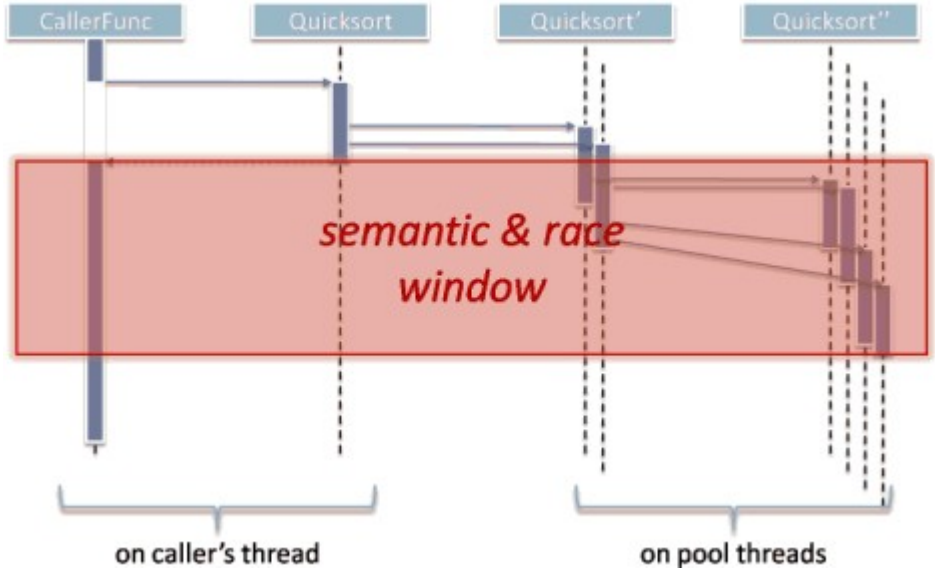

Figure 1

The failure to join actually causes two related but distinct problems:

- **Completion of side effects**: The caller expects that when the function returns the data in the range is sorted.
- **Synchronization**: The caller expects Quicksort won't still be accessing the data in the range, which would race with the caller's subsequent uses of that same data.

Remember, to the caller this appears to be a synchronous method. The caller expects all side effects to be completed, and has no obvious way to wait for them to complete if they're not.

Fortunately, the fix is easy: Just make sure that each invocation of Quicksort not only spawns the subrange sorting in parallel, but also waits for the spawned work to complete. To illustrate the idea, the following corrected example will use a **future** type along the lines of **futures** available in Java and in C++0x, but any kind of task handle that can be waited for will do:

```
// Example 3: Parallelized quicksort, take 2 (fixed)
void Quicksort( Iter first, Iter last ) {
  // If the range is small, synchronous sort is usually faster.
  if( distance( first, last ) < limit ) {
    OtherSort( first, last );
```

```
  return;
 }
 // 1. Pick a pivot position somewhere in the middle.
 // Move all elements smaller than the pivot value to
 // the pivot's left, and all larger elements to its right.
 Iter pivot = Partition( first, last );
 // 2. Recurse to sort the subranges. Run the first
 // asynchronously, while this thread continues with
 // the second.
 future fut =  pool.run(  [=]{ Quicksort( first, pivot ); }  );
 Quicksort( pivot, last );
 // 3. Join to ensure we don't return until the work is complete.
 fut.wait();
}
```

As illustrated in Figure 2, just making parent tasks wait for their subtasks makes the execution nicely structured, bounded, and deterministic. We've implemented a barrier where everyone including the caller waits before proceeding, and thus returned the desired amount of synchronicity:

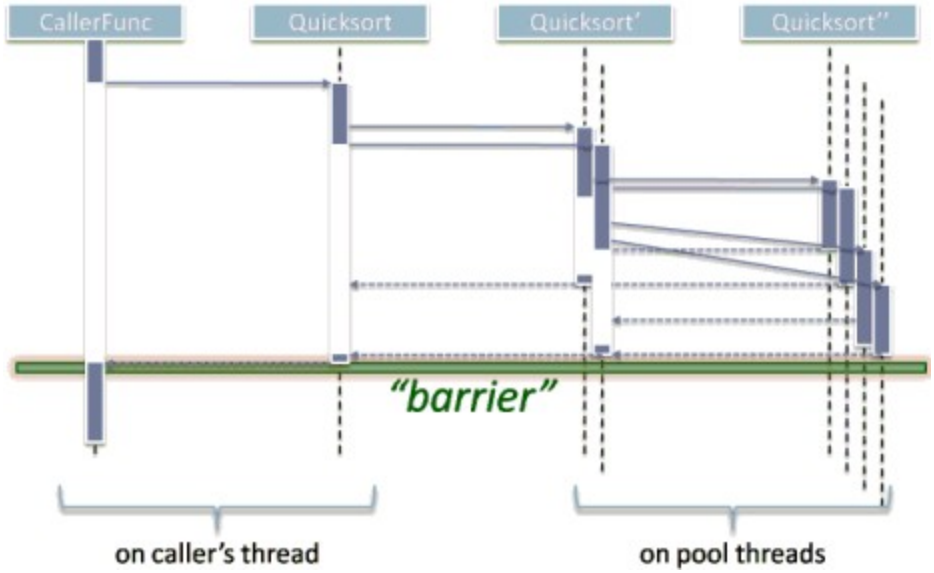

Figure 2

- **Completion of side effects**: We guarantee that when the function returns, it has done its full job and the range is sorted.
- **Synchronization**: After the function returns, some part of its work won't still be accessing the data, and so won't invisibly race with the caller's subsequent uses of the data.

Summary

Table 2 illustrates the differences between structured and unstructured lifetimes in the form of a little artsy photo sequence. In the following description

notice how consistently we use synonyms for our four keywords: local, nested, bounded, and deterministic.

Structured	Unstructured: What You Buy	Unstructured: What You Eat
		

Table 2

- **Structured**: As illustrated on the left side of Table 2, structured lifetimes are like Russian dolls. They nest cleanly, so that each doll fits entirely inside the next larger one. As the program is assembled and executed, each subgroup can be manipulated as a unit. When fully assembled, every individual part is nicely encapsulated and in a well-known place, easy to understand and reason about.
- **Unstructured (initial intent)**: When we first buy into unstructured lifetimes, we think we're buying into the middle picture of raw pasta. We start out with a manageable handful of stiff little independent and parallel lifetimes that don't twist or cross very much. Life is good, at least in the initial PowerPoint design
- **Unstructured (in practice)**: Unfortunately, as the final picture shows, we end up with something quite different. Software under maintenance behaves a lot like pasta: Once it's cooked and combined with other ingredients, the unstructured threads don't stay easy to grasp and easy to separate, but rather get intertwined and easier to break. What we actually wind up with ends up being quite slippery -- in the worst case, "spaghetti code."

Just because some work is done concurrently under the covers, behind a synchronous API call, doesn't mean there isn't any sequential synchronization with the caller. When a synchronous function returns to the caller, even if it did internal work in parallel it must guarantee that as much work as is needed is fully complete, including all side effects and uses of data.

Where possible, prefer structured lifetimes: ones that are local, nested, bounded, and deterministic. This is true no matter what kind of lifetime we're considering, including object lifetimes, thread or task lifetimes, lock lifetimes, or any other kind. Prefer scoped locking, using RAII lock-owning objects (C++, C# via **using**) or scoped language features (C# **lock**, Java **synchronized**). Prefer scoped tasks, wherever possible, particularly for divide-and-conquer and similar strategies where structuredness is natural. Unstructured lifetimes can be perfectly appropriate, of course, but we should be sure we need them because they always incur at least some cost in each of code complexity, code clarity and maintainability, and run-time performance. Where possible, avoid slippery spaghetti code, which becomes all the worse a nightmare to build and maintain when the lifetime issues are amplified by concurrency.

Acknowledgment

Thanks to Scott Meyers for his valuable feedback on drafts of this article.

Notes

[1] E. Dijkstra. "Go To Statement Considered Harmful" (Communications of the ACM, 11(3), March 1968). Available online at search engines everywhere.

[2] Here is one example of why deterministic destruction/dispose matters for performance: In 2003, the microsoft.com website was using .NET code and suffering performance problems. The team's analysis found that too many mid-life objects were leaking into Generation 2 and this caused frequent full garbage collection cycles, which are expensive. In fact, the system was spending 70% of its execution time in garbage collection. The CLR performance

team's suggestion? Change the code to **dispose** as many objects as possible before making server-to-server calls. The result? The system spent approximately 1% of its time in garbage collection after the fix.

[3] C# mistakenly called the finalizer the "destructor," which it is not, and this naming error has caused no end of confusion.

[4] E. Lee. "The Problem with Threads" (University of California at Berkeley, Electrical Engineering and Computer Sciences Technical Report, January 2006). http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-1.pdf.

[5] The new parallel task runtimes (e.g., Microsoft's Parallel Patterns Library and Task Parallel Library, Intel's Threading Building Blocks) are heavily optimized for structured task lifetimes, and their interfaces encourage structured tasks with implicit joins by default. The resulting performance and determinism benefits are some of the biggest improvements they offer over older abstractions like thread pools and explicit threads. For example, knowing tasks are structured means that all their associated state can be allocated directly on the stack without any heap allocation. That isn't just a nice performance checkmark, but it's an important piece of a key optimization achievement, namely driving down the cost of unrealized concurrency -- the overhead of expressing work as a task (instead of a synchronous function call), in the case when the task is not actually executed on a different thread -- to almost nothing (meaning on the same order as the overhead of an ordinary function call).

[6] The key to avoiding deadlock is to acquire locks in a deterministic and globally recognized order. Although releasing locks in reverse order is not as important for avoiding deadlock, it is usually natural. The key thing for keeping lock lifetimes structured is to release any locks the function took at least by the end of the function, which makes code self-contained and easier to reason about. Even when using a technique like hand-over-hand locking where lock release is not in reverse order of lock acquisition, locking should still be scoped so that all locks a function took are released by the time the function returns. See also H. Sutter. "Use Lock Hierarchies to Avoid Deadlock" (Dr. Dobb's Journal, 33(1), January 2008). Available online at http://www.ddj.com/hpc-high-performance-computing/204801163.

[7] H. Sutter. "Effective Concurrency: Avoid Exposing Concurrency -- Hide It Inside Synchronous Methods" (Dr. Dobb's Digest, October 2009). Available online at http://www.ddj.com/go-parallel/article/showArticle.jhtml?articleID=220600388 .